

Factor comparison methodology

Updated: 2026-06-24

This document explains, step by step, how `run_model_comparison.py` compares factor sets in the current codebase. It is written from the implementation as it exists now, including the updated brute-force machine-learning backtest logic.

The short version is this:

- 1 The comparison harness loads one shared market-data panel.
- 2 It resolves each named prerun into a set of factor records.
- 3 It keeps only factors that can actually calculate a non-empty signal on the

chosen panel.

- 1 It runs several comparison tracks on those usable factors:
 - single-factor IC diagnostics,
 - factor-library analytics,
 - brute-force ML combination and an IS/OOS vector backtest,
 - optionally, the full downstream agentic fund pipeline.
- 1 It writes machine-readable CSV/JSON outputs, figures, a Markdown report, and

a notebook under `data/comparisons/<comparison_id>/`.

The core fairness invariant is that the panel, dates, universe, split, model list, model hyper-parameters, and backtest settings are shared across preruns. The intended variable is the factor set contributed by each prerun. Forecast horizons are shared when `--horizon` is explicitly supplied; otherwise the brute-force combined horizon can be derived per prerun from its factors' `prediction_horizon` metadata.

Source map

The comparison path is spread over a small number of modules:

File	Role
<code>run_model_comparison.py</code>	CLI entry point and orchestration.
<code>quant_fund_agent/comparison/config.py</code>	Comparison configuration, date split logic, model resolution, provider overrides.
<code>quant_fund_agent/comparison/factors.py</code>	Prerun/seed factor resolution, shared panel loading, factor signal cache, usability filtering.
<code>quant_fund_agent/comparison/ic.py</code>	Single-factor IC evaluation.
<code>quant_fund_agent/comparison/analytics.py</code>	Diversity, redundancy, deflation, and model-view diagnostics.
<code>quant_fund_agent/comparison/bruteforce.py</code>	Brute-force ML feature construction, fitting, prediction, and ensemble logic.
<code>quant_fund_agent/comparison/vector_backtest.py</code>	Current vectorised IS/OOS backtest for brute-force model predictions.
<code>quant_fund_agent/comparison/standardize.py</code>	Shared per-underlying time-series standardisation used by IC, analytics, and brute-force ML.
<code>quant_fund_agent/backtesting/positions.py</code>	Signal z-scoring and position mapping primitives.

File	Role
quant_fund_agent/backtesting/strategy_backtester.py	Portfolio metric calculation and shared backtest metrics.
quant_fund_agent/backtesting/data_loader.py	Forward-return calculation.
quant_fund_agent/backtesting/engine.py	Cross-sectional IC calculation.
quant_fund_agent/data/panel.py	Provider-neutral market-data panel loading.
quant_fund_agent/data/providers/yfinance.py	yfinance OHLCV provider.
quant_fund_agent/data/cache.py	Per-symbol parquet cache for API provider data.
quant_fund_agent/data/universe.py	Universe preset and explicit ticker resolution.
quant_fund_agent/modeling/catalog.py	Fittable model catalog used by brute-force ML.
quant_fund_agent/comparison/report.py	Output tables, figures, report, and notebook generation.

Terminology

Panel:

A dictionary of wide data frames:

```
{
  "open": DataFrame(index=timestamps, columns=tickers),
  "high": DataFrame(index=timestamps, columns=tickers),
  "low": DataFrame(index=timestamps, columns=tickers),
  "close": DataFrame(index=timestamps, columns=tickers),
  "volume": DataFrame(index=timestamps, columns=tickers),
  ...
}
```

For yfinance equity data, the raw provider supplies OHLCV. The panel layer can also synthesize vwap from $(high + low + close) / 3$ and returns from `close.pct_change()`.

Prerun:

A named batch of generated researcher factors. Modern preruns live under `data/workspaces/<workspace>/preruns/<prerun>/`. Legacy preruns can also live under `data/factors/preruns/<prerun>/`. The compatibility layer resolves a bare name such as `sp100-5.4-mini` to the correct factor database.

Seed, main, or seeds:

Special comparison aliases for the seed factor library at `data/factors/factor_db.json`. Passing `seed`, `main`, or `seeds` as a prerun name means "compare the seed library by itself".

Usable factor:

A factor is usable for the chosen comparison data if:

- 1 its factor record is in the selected factor database,
- 2 its Python class is registered and importable,
- 3 `factor.calc(panel)` runs without raising,
- 4 the produced signal contains at least one finite value.

IS and OOS:

In-sample (IS) rows are used for fitting brute-force models. Out-of-sample (OOS) rows are held out for evaluation. With `--split-date 2024-06-01`, IS means timestamps strictly before 2024-06-01; OOS means timestamps on or after 2024-06-01.

Target or resolved horizon:

The forward-return horizon that the brute-force model tries to predict. If `--horizon 6` is supplied, the target for each (timestamp, ticker) row is the six-bar future return for that ticker. If `--horizon` is omitted, the brute-force track can resolve the horizon from factor `prediction_horizon` metadata.

Holding period:

The number of bars for which each target position tranche remains alive in the brute-force vector backtest. If omitted, it defaults to the resolved forecast horizon. In the current implementation this controls the rolling "book" used for one-bar mark-to-market PnL.

Book:

The live position after layering current and recent target positions. If the holding period is six, the book at bar `t` is the rolling average of the most recent six target-position rows.

1. CLI and configuration

The run starts in `run_model_comparison.py`. The CLI selects:

- which preruns to compare,
- which tracks to run,
- which market-data provider and universe to use,
- the in-sample/out-of-sample split,
- the prediction horizon,
- the holding period,
- the model list,
- speed knobs such as ticker caps, bar caps, and training-row subsampling,
- output directory and checkpoint behavior.

The important CLI arguments for the SP100/yfinance use case are:

```
--preruns
--provider yfinance
--asset-class equity
--frequency 1d
--universe-preset sp100
--data-start 2018-01-01
--data-end 2026-06-24
--split-date 2024-06-01
--horizon <bars>
--holding-period <bars>
```

The current date here is 2026-06-24, so using `--data-end 2026-06-24` means "through today" for the current run context.

The CLI values are collected into a `ComparisonConfig`. That config object is then passed down through the comparison modules. The config is also written to the output folder as `config.json`, which is the best first file to inspect when you later want to confirm exactly what a report used.

Track switches

By default the comparison can run:

- IC diagnostics,
- analytics diagnostics,

- brute-force ML,
- downstream agentic fund.

The user can disable tracks with:

```
--no-ic
--no-analytics
--no-bruteforce
--no-downstream
```

For an offline comparison of factor quality and brute-force ML on yfinance data, `--no-downstream` is usually the practical choice because the downstream track can spend LLM calls.

Seed inclusion semantics

There are two separate ways the seed library can enter a comparison:

- 1 Use `seed`, `main`, or `seeds` as a prerun name. This compares the seed

library as its own row in the report.

- 1 Use `--include-seeds`. This prepends the seed factors to every non-seed

prerun's factor set before usability filtering and model fitting.

These answer different questions:

- `--preruns seed, sp100-5.4-mini, sp100-4o-mini` asks: "How does the seed library alone compare to each generated library alone?"
- `--preruns sp100-5.4-mini, sp100-4o-mini --include-seeds` asks: "How does each generated library perform when augmented with the seed library?"
- Combining both includes a seed-only baseline row and generated-plus-seed rows.

2. Data-provider override and environment setup

Before loading data, `run_model_comparison.py` applies data-provider overrides from the CLI. These values are made visible through config and environment variables so that both direct in-process code and downstream subprocess-style paths see the same data selection.

For yfinance SP100 daily equity data, the relevant choices are:

```
provider:      yfinance
asset_class:   equity
frequency:    1d
universe_preset: sp100
start:        2018-01-01
end:          2026-06-24
```

The actual panel load is centralized in:

```
quant_fund_agent/comparison/factors.py::load_panel_cached
```

That function builds a data override dictionary from the comparison config and calls:

```
quant_fund_agent.data.load_panel(...)
```

It then injects the resulting panel into `quant_fund_agent.modeling.service`'s module-level panel cache and clears the signal cache. This is important for consistency: the comparison harness and any model-evaluation helper reuse the same active panel instead of each path silently loading a different data set.

3. yfinance panel construction

When `--provider yfinance` is selected, the provider path is:

```
data.panel.load_panel
```

```

-> get_provider(settings)
-> YFinanceProvider(settings)
-> ApiProvider.load
-> cached_fetch
-> YFinanceProvider._fetch

```

The yfinance provider:

- 1 Resolves the configured universe.
- 2 Translates canonical symbols to Yahoo symbols where needed.
- 3 Calls `yf.download(...)` with:
 - `start`,
 - `end`,
 - `interval`,
 - `auto_adjust=True`,
 - `group_by="ticker"`.
- 1 Reshapes the downloaded frame into per-symbol OHLCV frames.
- 2 Returns the data under canonical ticker names.

The cache layer stores one parquet file per (`provider`, `asset_class`, `frequency`, `symbol`) under the configured cache root. On a later run it reuses symbols whose cached range covers the requested start/end window. Missing or insufficiently covered symbols are refetched.

Universe resolution

For an API provider, the universe comes from one of:

- 1 an explicit `--tickers` list,
- 2 a bundled `--universe-preset`, such as `sp100`,
- 3 a point-in-time membership universe if configured in data settings.

For `--universe-preset sp100`, the code reads the bundled static text file `quant_fund_agent/data/universes/sp100.txt`. That is a static preset, not a point-in-time constituent history. Therefore, "all SP100 tickers when available" means:

- the provider attempts to load the preset tickers,
- symbols with available Yahoo data contribute columns,
- symbols with partial histories have missing values for unavailable dates,
- later factor calculations and finite-value masks decide which rows are usable.

For strict survivorship-bias-free index membership, the data layer supports a separate membership mode, but the SP100 preset itself is static.

Field availability for OHLCV factors

yfinance supplies:

`open`, `high`, `low`, `close`, `volume`

The panel layer can synthesize:

`vwap`, `returns`

Therefore factors that only need OHLCV, `vwap`, or `returns` can run on the yfinance panel. Factors requiring unavailable microstructure or other non-OHLCV fields should fail the usability pass and be excluded from that comparison.

4. Optional panel restriction

The comparison has two date-selection concepts:

- 1 The provider data range, controlled by `--data-start` and `--data-end`.
- 2 The IS/OOS split inside that loaded range.

With `--split-date`, the full provider date range is loaded. The split is then represented by masks over that full index.

With `--train-months` and `--oos-months`, `load_panel_cached` restricts the loaded panel to the union of the requested train and OOS calendar windows. This keeps every track focused on those calendar windows.

With `--max-bars`, the panel can be uniformly subsampled by stride after loading. This is only a speed knob. It changes the data seen by all tracks and should be used deliberately.

5. Split construction

The split masks are computed in `ComparisonConfig.split_masks(index)`.

Exact split date

For:

```
--split-date 2024-06-01
```

the masks are:

```
IS: index < 2024-06-01
OOS: index >= 2024-06-01
```

This is the split intended by the SP100 comparison request:

```
2018-01-01 <= timestamp < 2024-06-01      in-sample
2024-06-01 <= timestamp <= 2026-06-24     out-of-sample
```

Because daily market data may not have a bar exactly on 2024-06-01, the rule is still exact: every timestamp before the cutoff is IS and every timestamp on or after the cutoff is OOS.

For downstream compatibility, `run_model_comparison.py` also converts the split date into an OOS tail ratio:

```
oos_split_ratio = number_of_oosBars / total_number_ofBars
```

That ratio is needed because the downstream pipeline is ratio-based.

Calendar month windows

If `--train-months` and `--oos-months` are both provided, each is parsed as a comma-separated month list or inclusive month range. Example:

```
--train-months 2024-06:2024-08 --oos-months 2024-09
```

The train and OOS masks must be disjoint. This mode is useful for short month-specific experiments.

Tail-ratio fallback

If no split date or calendar windows are supplied, the split falls back to:

```
cut = int(n_timestamps * (1 - oos_split_ratio))
IS = first rows before cut
OOS = tail rows from cut onward
```

The default OOS ratio is 0.2.

6. Factor-set resolution

Each named prerun is resolved into factor records by:

```
quant_fund_agent/comparison/factors.py::prerun_factor_records
quant_fund_agent/comparison/factors.py::prerun_factor_ids
```

For a normal generated prerun, the comparison reads the prerun's `factor_db.json` and returns its researcher factors.

For seed, main, or seeds, the comparison reads the seed factors from `data/factors/factor_db.json`.

If `--include-seeds` is set, the seed factors are prepended to the generated prerun's researcher factors. IDs are de-duplicated in order, so a repeated factor ID only appears once.

The factor name map is built separately for presentation, so report tables can show human-readable names where possible while still using factor IDs as stable keys.

7. Factor discovery and usability filtering

Before evaluating factors, the comparison imports and discovers factor classes. The registry maps:

```
factor_id -> Python factor class
```

Usability filtering then loops through each factor ID for each prerun and tries to compute its signal on the shared panel.

Conceptually:

```
for factor_id in selected_factor_ids:
    cls = get_factor_class(factor_id)
    if cls is missing:
        drop factor
        continue

    signal = cls().calc(panel)
    if calculation raises:
        drop factor
        continue

    if signal is entirely NaN/non-finite:
        drop factor
        continue

    keep factor
```

The calculated signals are cached. Later tracks reuse the cached factor signals instead of recalculating the same factor repeatedly.

The usability pass is especially important for provider comparisons. A factor can exist in the database but still be impossible to evaluate on yfinance if it requires fields the yfinance panel does not provide. Pure OHLCV factors should be admitted, provided their code runs and returns finite values on at least part of the selected panel.

The output `usability.json` records, per prerun:

- how many factors were selected,
- how many were usable,
- which IDs were usable,
- which IDs were dropped.

8. Track A: single-factor IC diagnostics

The IC track asks:

"If this factor is used by itself, does its value predict the same underlying's own future return?"

That is a deliberately different question from the old cross-sectional IC question. The current default is non-cross-sectional. It does not rank AAPL against MSFT against the rest of the universe at each date. Instead, it treats each factor as a directional time-series signal per underlying, standardizes it per underlying, pools the valid (timestamp, underlying) observations, and computes one Spearman rank correlation against the matching forward-return vector.

The implementation lives in `quant_fund_agent/comparison/ic.py`. The default path is selected when:

```
cfg.fit_standardize == "per_underlying"
```

which is the default. The legacy cross-sectional IC path is still available only when:

```
--fit-standardize cross_sectional
```

In that legacy mode the code delegates to `backtesting.engine.backtest_factor`.

The current default IC horizon grid remains:

```
1, 6, 60
```

If `--horizon H` is supplied, the grid becomes:

```
1, H, 60
```

Each factor can also have its own `prediction_horizon` in the factor database. The comparison reads those metadata values through `comparison.factors.prediction_horizons(...)`. When available, the IC row also includes the factor's own-horizon fields:

```
horizon_own
ic_own
icir_own
ic_hit_own
```

This matters when factors in the same library were generated for different forecast horizons.

Forward returns

Forward returns are computed as:

```
forward_return_h[t, ticker] =
    close[t + h, ticker] / close[t, ticker] - 1
```

This comes from `quant_fund_agent/backtesting/data_loader.py::forward_returns`.

Default per-underlying pooled IC

For a factor signal `sig` and a horizon `h`, the default IC code does:

- 1 Reindex the signal to the `close` panel's timestamps and columns.
- 2 Standardize each underlying's signal over time with

```
per_underlying_zscore(sig).
```

- 1 Flatten the standardized signal matrix into one vector:

```
x = zscored_signal.to_numpy().ravel()
```

- 1 Compute the matching `h`-bar forward returns and flatten them in the same

timestamp-major order:

```
y = forward_returns(close, horizon=h).to_numpy().ravel()
```

- 1 Keep finite paired observations.
- 2 Compute one Spearman rank correlation:

```
IC_h = SpearmanRankCorr(x, y)
```

With one ticker this is exactly the Spearman correlation between that ticker's factor-value history and its own future-return history. With many tickers, it pools those per-underlying histories into one larger vector. The universe helps by supplying more observations, not by creating a cross-sectional ranking at each timestamp.

Stability and hit-rate diagnostics

The current per-underlying IC row contains:

- mean IC,
- an IC information ratio,
- IC hit rate,
- an approximate IC t-statistic,

- number of valid paired observations.

For the default per-underlying path, the IC information ratio is not the old standard deviation of timestamp-level cross-sectional ICs. Instead, the pooled vector is split into contiguous blocks, an IC is calculated per block, and the IR is:

```
mean(block_ICs) / std(block_ICs)
```

The hit rate is also time-series directional:

```
share(sign(signal_value) == sign(forward_return))
```

over finite non-zero paired observations.

Legacy cross-sectional IC

If `--fit-standardize cross_sectional` is selected, the IC track keeps the older cross-sectional definition:

```
IC[t] = SpearmanRankCorr(signal[t, :], forward_return_h[t, :])
```

That mode requires enough valid assets per timestamp. It can be useful when the research question is explicitly "does the factor rank the universe correctly on this date?", but it is no longer the default comparison lens.

IC summary by prerun

After all factors are evaluated, the comparison summarizes each prerun:

- mean absolute IC by horizon,
- median absolute IC by horizon,
- mean absolute ICIR by horizon,
- share of factors with $\text{abs}(\text{IC}) > 0.02$,
- if own-horizon values exist, mean/median own-horizon IC and ICIR,
- best factor at horizon six if horizon six is available.

Important nuance: IC is descriptive, not IS/OOS

The single-factor IC track does not fit a model and does not currently report separate IS and OOS IC columns. It evaluates over the loaded panel passed to the track.

For a `--split-date SP100` run with `--data-start 2018-01-01` and `--data-end 2026-06-24`, this means the IC diagnostics are full-sample diagnostics over that loaded date range.

For `--train-months/--oos-months`, the panel has already been restricted to the train union OOS calendar windows, so IC is calculated on that restricted union.

The IS/OOS validation distinction is primarily in the brute-force ML track.

9. Track B: factor-library analytics

The analytics track asks:

"What is the structure of this factor library, independently of one exact trading strategy?"

It has three main parts:

- 1 diversity and redundancy,
- 2 multiple-testing deflation,
- 3 model-view feature importance.

Analytics feature matrix

The analytics code builds a feature matrix from usable factor signals.

For each factor:

- 1 Retrieve the cached factor signal.
- 2 Align it to the close-price grid.
- 3 Standardize it according to `cfg.fit_standardize`.
- 4 Flatten the timestamp x ticker grid into one long vector.

The result is a matrix:

```
X: rows = timestamp/ticker observations  
   columns = factors
```

The default standardization is:

```
cfg.fit_standardize == "per_underlying"
```

In that mode each factor is z-scored per underlying over time using `comparison.standardize.per_underlying_zscore`. This is non-cross-sectional and works even with a one-ticker panel. A factor column in X means "this factor's standardized value through time, pooled across underlyings."

The legacy cross-sectional feature matrix is still available with:

```
--fit-standardize cross_sectional
```

Only in that mode does analytics call `normalise_factor_signals` to z-score signals across tickers at each timestamp.

Rows are optionally subsampled by `--analytics-max-rows` for speed.

Diversity and redundancy

The diversity analysis computes the factor correlation matrix:

```
corr = X.corr()
```

From that matrix it calculates:

- `mean_abs_corr`: average absolute pairwise factor correlation,
- per-factor mean absolute correlation,
- per-factor max absolute correlation,
- clusters under a configured absolute-correlation threshold,
- eigenvalue-based effective number of factors,
- redundancy score.

The effective number of factors uses the participation-ratio idea:

```
effective_n = (sum(eigenvalues) ** 2) / sum(eigenvalues ** 2)
```

If all factors are independent and equally informative, `effective_n` approaches the raw factor count. If many factors are duplicates or near-duplicates, `effective_n` is lower.

The redundancy score is:

```
redundancy = 1 - effective_n / n_factors
```

Higher redundancy means more of the library is explaining the same variation.

Because the default feature matrix is now pooled per-underlying/time-series, the correlation answers:

"Do these two factors tend to move similarly over the pooled observation set?"

It is not primarily asking:

"Do these two factors rank the cross-section similarly on each date?"

That older interpretation applies only when `--fit-standardize cross_sectional` is selected.

Multiple-testing deflation

The deflation diagnostic uses the IC results at the target horizon. It asks:

"If this prerun tried many factors, how much of the best observed IC might just be the luck of many tests?"

This analytics target horizon is `cfg.target_horizon`: six by default, or the explicit value from `--horizon`. It is not the per-prerun derived combined horizon used later inside the brute-force track when `--horizon` is omitted.

The code estimates an expected maximum null IC roughly as:

```
expected_luck_ic = sqrt(2 * log(k)) / sqrt(n_obs)
```

where:

- `k` is the number of tested factors,
- `n_obs` is the number of IC observations.

It also computes a deflated best t-statistic:

```
deflated_best_t = best_ic * sqrt(n_obs) - sqrt(2 * log(k))
```

This is not a full academic deflated Sharpe implementation. It is a compact multiple-testing haircut meant to make "best factor" claims less naive.

Model-view feature importance

The model-view diagnostic asks:

"If a simple supervised model tries to explain the underlying's own target-horizon return from these factors, which factors does it use most?"

The target is:

```
y = forward_return_target_horizon
```

As with deflation, this means `cfg.target_horizon`, not the brute-force per-prerun derived horizon.

The feature matrix is the analytics matrix described above, so it is also per-underlying/time-series by default. Missing factor values are filled with zero after standardization. Under the default standardizer, zero means "at the underlying's fitted mean"; under the legacy cross-sectional standardizer, zero means "at the cross-sectional mean for that timestamp."

The default importance models are:

```
lasso  
gradient_boosting
```

For linear models, the code extracts absolute coefficients. For tree/boosting models, it extracts feature importances. Importances are normalized so they sum to one within each fitted diagnostic model.

This is a diagnostic lens, not the same as the brute-force IS/OOS backtest. It helps identify whether the library's signal appears concentrated in a few factors or spread across many.

10. Track C: brute-force ML comparison

The brute-force ML track is the central IS/OOS comparison. It asks:

"If we give the same machine-learning model access to each prerun's usable factor set, which factor set produces better out-of-sample trading behavior?"

In the current code this is mostly not a cross-sectional comparison. Each model learns a mapping from factor values to the same underlying's own future return. The prediction is one directional combined signal per (`timestamp`, `underlying`). The backtest then trades that signal on that underlying. No cross-sectional ranking is used by the default brute-force path.

For each prerun:

- 1 Build a supervised learning data set from that prerun's usable factor

signals.

- 1 Fit each selected model on IS rows only.
- 2 Predict a combined alpha signal for the full panel.
- 3 Convert the combined signal into positions.
- 4 Run the vectorised IS/OOS backtest.
- 5 Optionally build an ensemble across the fitted models.

The supervised learning target

The target for a model is each underlying's own forward return at the resolved combined-signal horizon:

```
y[t, ticker] = close[t + resolved_horizon, ticker] / close[t, ticker] - 1
```

If `--horizon 6` is supplied, the resolved horizon is explicitly six bars for every prerun. If `--horizon` is omitted, the current code derives each prerun's combined horizon from the `prediction_horizon` metadata of its usable factors. The aggregation is controlled by:

```
--combined-horizon-agg mode|median|max|min|explicit
```

The default is `mode`: use the most common factor horizon in that prerun, with ties broken toward the smaller horizon. The resolved value is clamped to the loaded panel length so it cannot create an all-NaN target.

This is an important change to the fairness story:

- If you want every prerun forced onto exactly the same target horizon, pass

```
--horizon.
```

- If you omit `--horizon`, the comparison lets each prerun's factor metadata

determine the combined-signal horizon, which can be appropriate when factor horizon is part of what the research model produced.

This target is used for fitting the model. It is not directly multiplied by the position in the current vector backtest.

Feature construction

For a prerun with F usable factors:

- 1 Each factor signal is aligned to the close grid.
- 2 Infinite values are replaced with NaN.
- 3 Each factor becomes one feature column.

There are two feature-standardization modes.

Per-underlying standardization

This is the default for brute-force ML.

For each factor and ticker, the code calculates mean and standard deviation using IS rows only:

```
mean_j = mean(signal[IS, ticker_j])
std_j = std(signal[IS, ticker_j])
```

Then the full series is standardized with those IS statistics:

```
z[t, ticker_j] = (signal[t, ticker_j] - mean_j) / std_j
```

This avoids using OOS distribution information to scale OOS features.

This is also the main change that eliminates most cross-sectional dependence. The standardization is time-series per underlying: AAPL is scaled against AAPL's own IS history, MSFT against MSFT's own IS history, and so on.

Cross-sectional standardization

If `--fit-standardize cross_sectional` is selected, each factor is normalized cross-sectionally at each timestamp using the shared `normalise_factor_signals` helper.

This is now the opt-in legacy mode. It emphasizes relative cross-sectional ranking at each bar and requires a multi-name cross-section to be meaningful.

Flattening into model rows

The feature tensor has shape:

`timestamps x tickers x factors`

It is flattened to:

`rows x factors`

where each row is one `(timestamp, ticker)` observation.

The flattening order is timestamp-major. In other words, all tickers for the first timestamp appear first, then all tickers for the second timestamp, and so on.

The target vector is flattened in the same order.

Training rows

The model fit mask keeps rows where:

- the timestamp belongs to the IS mask,
- the target `y` is finite.

Factor feature NaNs are replaced with zero for model fitting and prediction. After standardization, zero means "neutral/missing/average" rather than an extreme value.

If `--train-sample-frac` is below one, the fit rows are randomly subsampled with a fixed RNG seed. This accelerates large panels while keeping the backtest itself on the full panel.

The `--fast` preset sets a lower default training fraction and lighter tree model parameters unless explicitly overridden.

Fit scope: pooled or per-underlying

The brute-force code supports two fit scopes.

Pooled

In pooled mode, one estimator is fit using all valid IS `(timestamp, ticker)` rows.

The estimator learns one common mapping:

`factors -> forward return`

across the pooled observation set.

This is pooling, not cross-sectional ranking. The rows from all underlyings are combined to give the model more training examples, but each row's target is still that row's own underlying return.

It then predicts every row in the full panel, including OOS rows. The predictions are reshaped back to:

`timestamps x tickers`

This prediction matrix is the combined signal.

Per-underlying

In per-underlying mode, the code trains a separate estimator for each ticker.

For ticker `j`, it keeps only rows belonging to that ticker and fits:

`factors_for_ticker_j -> forward_return_for_ticker_j`

The implementation requires a minimum number of valid IS rows per ticker. If a ticker does not have enough valid training rows, its prediction column remains NaN.

Per-underlying mode can capture ticker-specific relationships, but it is more data-hungry.

Both fit scopes produce the same output shape: one combined signal value per `(timestamp, ticker)`.

Model catalog

The fittable model types come from `quant_fund_agent/modeling/catalog.py`.

The available model list can include:

- `linear_regression`,
- `ridge`,
- `lasso`,
- `elastic_net`,
- `random_forest`,
- `gradient_boosting`,
- `xgboost`,
- `lightgbm`.

Native optional models such as `xgboost` and `lightgbm` are included only if their packages can be imported in the current environment.

Every model is wrapped in a standard sklearn pipeline that standardizes features and standardizes the target before fitting. Linear regularized models use cross-validation internally for their regularization settings.

The comparison deliberately excludes the legacy static-weights baseline from the default brute-force fittable model list. This keeps the track focused on models that actually learn from the factor set.

Ensemble

If at least two individual model results exist and `--no-ensemble` is not set, the comparison creates an ensemble row.

For each model's combined signal:

- 1 Flatten the signal.
- 2 Compute its standard deviation.
- 3 Rescale the signal by that standard deviation.
- 4 Average the rescaled signals.

The ensemble signal is then sent through the same vector backtest as any individual model signal.

This keeps one model with larger raw prediction scale from dominating the ensemble simply because its predictions have larger units.

11. Current brute-force vector backtest

This section is the most important part of the current implementation, because the brute-force backtest logic has changed.

The old problematic pattern was effectively:

```
position[t] * h_bar_forward_return[t]
```

When adjacent h-bar returns overlap, that style can inflate annual return by roughly the horizon and Sharpe by roughly $\sqrt{\text{horizon}}$, because the same price movement is counted repeatedly.

The current `vector_backtest.py` avoids that by:

- 1 standardizing the combined model signal per underlying over time,
- 2 converting that signal to directional target positions per underlying,

- 3 layering target positions into a rolling tranche book,
- 4 marking that live book to market on each underlying's own one-bar forward return,
- 1 aggregating the resulting per-underlying PnL.

There is no default cross-sectional long-short construction in this backtest. Each column is treated as its own directional signal first. Cross-underlying aggregation happens after PnL is calculated.

Step 1: align the combined signal

The model's prediction matrix is reindexed to the close-price grid:

```
sig = sig.reindex_like(close)
```

The signal has the same conceptual shape as close prices:

```
timestamps x tickers
```

Step 2: z-score the signal over time

The backtest converts raw prediction magnitudes into comparable z-scores using:

```
zscore_over_time(signal, basis, window)
```

The basis is controlled by:

```
--position-zscore
```

Available bases:

Basis	Meaning
expanding	Expanding per-underlying z-score over time. This is the default.
rolling	Rolling per-underlying z-score over a fixed window.
full	Full-sample per-underlying z-score. Useful diagnostically but uses future distribution information.
none	Do not z-score; use raw signal values.

With the default expanding basis, each ticker is scaled against its own history up to the current row. It does not use future rows.

Step 3: map z-scores to target positions

The z-scored signal is converted to target positions using:

```
directional_positions(z, mode, threshold)
```

The mode is controlled by:

```
--position-mode
```

Available modes:

Mode	Position rule
threshold	Long if $z \geq \text{threshold}$, short if $z \leq -\text{threshold}$, flat otherwise.
sign	Long when signal is positive, short when signal is negative.
continuous	Use clipped signal strength between -1 and +1.

With default threshold mode:

```
target_pos[t, ticker] =
    +1 if z[t, ticker] >= threshold
    -1 if z[t, ticker] <= -threshold
    0 otherwise
```

These are target positions, not yet the final live book.

Step 4: create the rolling tranche book

Let:

`h = holding_period` if supplied else `resolved_horizon`

If `h == 1`, the book is just the target position:

```
book[t] = target_pos[t]
```

If `h > 1`, the book is:

```
book[t] = mean(target_pos[t], target_pos[t - 1], ..., target_pos[t - h + 1])
```

In pandas, the implementation is:

```
book = target_pos.rolling(h, min_periods=1).mean()
```

The economic interpretation is:

- every bar opens a new tranche sized $1 / h$,
- that tranche follows the current target position,
- it remains active for `h` bars,
- the live book is the sum, equivalently average, of active tranches.

Example with `h = 3` for one ticker:

Bar	Target position	Live book
t0	+1	+1.000
t1	+1	+1.000
t2	-1	+0.333
t3	0	0.000
t4	-1	-0.667

This is the crucial change: the holding period determines how long target positions remain in the book, but the book is still marked on one-bar returns.

Step 5: one-bar mark-to-market PnL

The backtest computes one-bar forward returns:

```
r1[t, ticker] = close[t + 1, ticker] / close[t, ticker] - 1
```

Then it computes PnL as:

```
pnl[t, ticker] = book[t, ticker] * r1[t, ticker]
```

This means a signal formed at bar `t` is evaluated on the price move from bar `t` to bar `t + 1`.

The resolved horizon is still used for model training and IC diagnostics. The holding period controls position persistence. The realized backtest PnL is always marked to market one bar at a time.

Step 6: aggregate across underlyings

There are two aggregation modes:

```
--aggregation portfolio
--aggregation per_underlying
```

Portfolio mode is the default. It averages PnL across tickers at each timestamp:

```
portfolio_return[t] = mean_tickers(pnl[t, :])
```

Then it calculates one set of portfolio metrics for IS and one for OOS.

Per-underlying mode calculates metrics separately for each ticker and then averages those metrics across tickers. It also reports the cross-sectional standard deviation of OOS Sharpe as `oos_sharpe_std`. Here "cross-sectional" only means a dispersion statistic across already-computed single-name metrics; it is not a cross-sectional ranking signal or long-short portfolio construction.

Portfolio mode answers, "What did the whole equal-weighted book do?"

Per-underlying mode answers, "What was the average behavior of a single-name strategy?"

Step 7: metric calculation

The current vector backtest reports:

- `is_ic`,
- `oos_ic`,
- `is_sharpe`,
- `oos_sharpe`,
- `is_ann_return`,
- `oos_ann_return`,
- `oos_max_drawdown`,
- `oos_hit_rate`,
- `oos_sharpe_std` in per-underlying aggregation mode.

Most portfolio metrics are calculated by the shared `_compute_metrics` helper in `strategy_backtester.py`.

That helper infers frequency from the timestamp index and annualizes accordingly. For daily yfinance data it should behave like a daily strategy; for intraday data it scales by the inferred number of bars per trading day.

The IC reported by `vector_backtest.py` is also non-cross-sectional by default. It uses `_per_underlying_ic`, which applies the same pooled per-underlying Spearman idea as the IC track:

```
IC = SpearmanRankCorr(
    per_underlying_zscore(combined_signal).ravel(),
    forward_returns(close, resolved_horizon).ravel()
)
```

Therefore:

- PnL uses one-bar returns and the tranche book,
- IC measures whether the signal's pooled per-underlying values line up with

resolved-horizon future returns.

Those two quantities answer different questions and both are useful.

12. What prevents OOS leakage in the brute-force track

The current brute-force track uses several safeguards:

- 1 The train mask only includes IS timestamps.
- 2 In default per-underlying feature standardization, means and standard

deviations are estimated on IS rows only.

- 1 Models are fit only on IS rows.
- 2 OOS rows are predicted by the already-fitted estimator.
- 3 Default position z-scoring uses an expanding per-underlying history, so

position scaling at a bar only uses information available up to that bar.

1 The vector backtest applies the same position and metric logic to IS and OOS.

2 PnL is marked on one-bar forward returns, avoiding overlapping h-bar return

inflation.

There are also caveats worth knowing:

1 If `--fit-standardize cross_sectional` is selected, the comparison re-enters

the legacy cross-sectional feature/IC mode. That is valid for a specific cross-sectional research question, but it is no longer the default.

1 If `--position-zscore full` is used, position scaling uses full-sample

distribution information. That is useful for diagnostics but not a clean OOS setting.

1 If `--horizon` is omitted, brute-force combined-signal horizons can differ by

prerun because they are derived from factor `prediction_horizon` metadata. Pass `--horizon` when the experiment requires a strictly identical forecast horizon across every prerun.

1 The yfinance provider uses `auto_adjust=True`, so prices are split/dividend

adjusted over the full downloaded history. That is standard in daily equity factor research but is not a point-in-time corporate-action feed.

1 The sp100 universe preset is a static list unless a point-in-time

membership source is configured separately.

1 The single-factor IC and analytics tracks are diagnostics over the loaded

panel, not separate IS/OOS validation tracks.

1 The default methods no longer need a cross-section to be defined, but a

broader universe still provides more pooled observations and a more diverse test of whether the factor logic generalizes across names.

13. Track D: optional downstream fund comparison

The downstream track is different from the offline tracks. It spends LLM calls and runs the agentic fund pipeline.

For each prerun it:

1 Composes a factor database for that prerun under the comparison output

directory.

1 Optionally includes seed factors if `--include-seeds` is set.

2 Points `FACTOR_DB_PATH` at the composed database.

3 Runs the Selector -> Architect -> Statistician pipeline for

`--n-strategies` attempts.

1 Persists approved strategies.

2 Runs the Portfolio Manager to allocate across approved strategies.

3 Writes strategy-level and portfolio-level summaries.

The downstream track uses `oos_split_ratio`. With `--split-date`, the comparison derives a ratio that corresponds to the number of OOS bars under that split.

Because the downstream path is a fuller agentic workflow, it is slower and less isolated than brute-force ML. It answers a different question:

"If the fund machinery is allowed to build strategies from this factor set, what does it approve and allocate to?"

The brute-force ML track answers the cleaner factor-library A/B question.

14. Outputs

All outputs go under:

data/comparisons/<comparison_id>/

The comparison writes:

File	Meaning
config.json	Exact comparison configuration and resolved preruns.
usability.json	Usable/dropped factors by prerun.
ic_results.csv	Per-factor IC metrics.
ic_summary.csv	Per-prerun IC summary.
analytics_diversity_summary.csv	Per-prerun diversity/redundancy summary.
analytics_diversity_factor.csv	Per-factor redundancy diagnostics.
analytics_modelview_summary.csv	Summary of diagnostic importance models.
analytics_importance.csv	Factor importances from diagnostic models.
bruteforce_results.csv	IS/OOS brute-force ML results by prerun/model.
downstream_results.csv	Optional downstream portfolio summaries.
downstream_strategies.csv	Optional downstream strategy-level rows.
factor_correlation_<prerun>.csv	Correlation matrix per prerun.
report.md	Human-readable summary report.
comparison.ipynb	Notebook for interactive inspection.
figures/*.png	Rendered comparison charts.

The report and figures are presentation layers over the CSV/JSON outputs. When in doubt, inspect config.json, usability.json, and bruteforce_results.csv first.

15. Checkpointing

The entry point writes checkpoints after major stages unless --no-checkpoint is used. This matters for long SP100 runs because factor calculation and model fitting can take time.

The checkpoints preserve intermediate results such as:

- usability,
- IC rows and summary,
- analytics rows,
- brute-force rows,
- downstream rows.

They make it easier to recover partial work if a later optional stage fails.

16. Recommended command for the current SP100 comparison

To compare the two generated SP100 preruns and include a seed-only baseline row:

```
./venv/bin/python run_model_comparison.py \  
--preruns seed,sp100-5.4-mini,sp100-40-mini \  
--no-downstream \  
--provider yfinance \  
--asset-class equity \  
--frequency 1d \  
--universe-preset sp100 \  
--data-start 2018-01-01 \  
--data-end 2026-06-24 \  
--split-date 2024-06-01
```

To compare each generated library with the seed factors included inside both generated libraries, use:

```
./venv/bin/python run_model_comparison.py \  
--preruns sp100-5.4-mini,sp100-40-mini \  
--include-seeds \  
--no-downstream \  
--provider yfinance \  
--asset-class equity \  
--frequency 1d \  
--universe-preset sp100 \  
--data-start 2018-01-01 \  
--data-end 2026-06-24 \  
--split-date 2024-06-01
```

To do both in one run:

```
./venv/bin/python run_model_comparison.py \  
--preruns seed,sp100-5.4-mini,sp100-40-mini \  
--include-seeds \  
--no-downstream \  
--provider yfinance \  
--asset-class equity \  
--frequency 1d \  
--universe-preset sp100 \  
--data-start 2018-01-01 \  
--data-end 2026-06-24 \  
--split-date 2024-06-01
```

In the last command:

- the seed row is seed-only,
- the sp100-5.4-mini row is generated factors plus seeds,
- the sp100-40-mini row is generated factors plus seeds.

If the thesis question is "did generated factors add value over the seed library?", the first or third command is usually the most informative.

17. How to read the brute-force result table

The most important output for the requested comparison is:

bruteforce_results.csv

Each row corresponds to:

prerun x model

plus optional ensemble rows.

Key columns:

Column	Interpretation
prerun	Factor set being evaluated.
model	ML model fitted to that factor set.
n_factors	Number of usable factors passed into the model.

Column	Interpretation
fit_scope	Pooled or per-underlying fitting.
is_ic	Composite signal IC on the IS period at the resolved/target horizon.
oos_ic	Composite signal IC on the OOS period at the resolved/target horizon.
is_sharpe	Backtested Sharpe on the IS period.
oos_sharpe	Backtested Sharpe on the OOS period.
is_ann_return	Annualized IS return from one-bar marked PnL.
oos_ann_return	Annualized OOS return from one-bar marked PnL.
oos_max_drawdown	OOS max drawdown.
oos_hit_rate	Share of positive OOS return bars.

For a clean comparison, focus more on OOS columns than IS columns. IS performance can show whether the model learned anything, but OOS performance is the actual validation target.

Useful signs:

- OOS Sharpe positive and not much lower than IS Sharpe: better generalization.
- IS Sharpe high but OOS Sharpe near zero or negative: likely overfit or unstable

factor relationship.

- OOS IC positive but OOS Sharpe weak: the directional signal may contain

information, but the position construction or return distribution may not monetize it well.

- High `n_factors` but low effective factor count in analytics: library may be

large but redundant.

18. How to read usability results

The first thing to check after a `yfinance` run is:

`usability.json`

If a factor is missing from the usable list, it usually means one of:

- 1 The factor ID is present in the database but its class is not registered.
- 2 The factor calculation raised an exception.
- 3 The `yfinance` panel does not contain a required input field.
- 4 The factor returned all NaNs on the requested universe/date range.
- 5 The ticker/date coverage was too sparse for that factor's rolling window.

For pure OHLCV seed factors, the expected behavior is that they should be usable on `yfinance` unless their implementation itself errors or the selected panel contains insufficient data for their lookback requirements.

19. What the comparison does not claim

The comparison is useful, but it should not be over-read.

It does not claim:

- that the SP100 static preset is survivorship-bias-free,
- that `yfinance` adjusted prices are a perfect point-in-time institutional feed,
- that single-factor IC summaries are OOS validation metrics,

- that the downstream agentic track is as controlled as brute-force ML,
- that a good OOS Sharpe in this research backtest includes full transaction

costs, borrow costs, market impact, or execution constraints.

It does claim:

- that factor sets are evaluated on the same loaded panel,
- that the brute-force ML models are fit on IS rows only,
- that OOS rows are held out from fitting,
- that the current brute-force PnL uses a one-bar marked tranche book rather

than overlapping h-bar return multiplication,

- that all generated output files can be traced back to the written

`config.json`.

20. End-to-end SP100 flow

For the specific requested setup:

```
provider:      yfinance
universe:      static sp100 preset
data start:    2018-01-01
split date:    2024-06-01
data end:      2026-06-24
preruns:       sp100-5.4-mini, sp100-40-mini, optionally seed/main
```

the actual flow is:

- 1 Resolve the CLI into `ComparisonConfig`.
- 2 Load the yfinance daily equity panel for the SP100 preset from
2018-01-01 through 2026-06-24, using cached parquet files where possible.
- 1 Synthesize vwap and returns if requested by factors.
- 2 Build split masks:
 - IS: dates before 2024-06-01,
 - OOS: dates on or after 2024-06-01.
- 1 Resolve each selected prerun into factor IDs.
- 2 Add seed factors if requested by `--include-seeds`.
- 3 Discover/import factor classes.
- 4 Calculate each factor on the shared yfinance panel.
- 5 Drop factors whose code is unavailable, whose calculation fails, or whose
signal is entirely non-finite.
- 1 Run full-sample single-factor IC diagnostics on the usable factors.
- 2 Run full-sample library analytics on the usable factors.
- 3 For each prerun and model:
 - build factor features,
 - build resolved-horizon forward-return labels,
 - train on IS rows,

- predict the full panel,
 - z-score predictions for position construction,
 - map predictions to target positions,
 - layer target positions into a holding-period book,
 - mark the book on one-bar forward returns,
 - report IS and OOS metrics separately.
- 1 Build ensemble rows if enabled and possible.
 - 2 Write all outputs to `data/comparisons/<comparison_id>/`.

That is the current comparison methodology implemented by the codebase.